Vídeo 6.3: Documentando a API com Swagger

Duração-alvo: 6 min | **Conceitos:** documentação, OpenAPI e Swagger UI | **O aluno sai sabendo:** descrever rotas em um arquivo OpenAPI e usar a página interativa para testá-las.

Antes de gravar: partir do código do vídeo 6.2, com o servidor rodando por node src/server.js. O arquivo src/openapi.json completo (todas as rotas) fica pronto no repositório; no vídeo, escrever só a parte mostrada abaixo.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “Imagine alguém querendo usar a sua API. Como essa pessoa descobre quais rotas existem, o que enviar e o que esperar? Perguntando para você toda vez? Para isso existe a documentação. Hoje vamos criar uma que, além de explicar, deixa a pessoa testar as rotas direto no navegador.”

[0:50] OpenAPI e Swagger

Na tela: um cardápio de restaurante ao lado de uma captura da página do Swagger UI.

Fala: “Existe um padrão mundial para descrever APIs, chamado OpenAPI. É um arquivo que lista as rotas, os métodos, o que cada uma recebe e o que devolve. E existe uma ferramenta, o Swagger UI, que lê esse arquivo e monta uma página interativa. Pense no cardápio de um restaurante: ele lista o que existe, o que cada prato traz, e ainda deixa você fazer o pedido.”

[1:45] Instalando e ligando a documentação

Na tela: no terminal, instalar o pacote. Depois, em src/app.js, importar a ferramenta e acrescentar a linha do /docs antes dos dois tratadores do final do arquivo.

```
npm install swagger-ui-express


const swaggerUi = require('swagger-ui-express');
 const documentacao = require('./openapi.json');

 app.use('/docs', swaggerUi.serve, swaggerUi.setup(documentacao));
```

Fala: “Instalo o swagger-ui-express. No app ponto js, importo a ferramenta e o arquivo openapi ponto json, que vamos escrever já. E adiciono uma linha: no endereço barra docs, mostre a documentação. Essa linha precisa vir antes do tratador de rota desconhecida. Senão, o barra docs seria tratado como uma rota que não existe.”

[2:45] Escrevendo o arquivo

Na tela: criar o arquivo src/openapi.json com o conteúdo abaixo, destacando cada parte conforme a explicação.

```
{
  "openapi": "3.0.3",
  "info": {
```

```
        "title": "API da Biblioteca Comunitária",
        "version": "1.0.0",
        "description": "API usada no curso API na Prática."
    },
    "paths": {
        "/livros": {
        "get": {
        "summary": "Lista os livros",
        "parameters": [
        {
        "name": "autor",
        "in": "query",
        "description": "Filtra pelo nome do autor",
        "schema": { "type": "string" }
        }
        ],
        "responses": {
        "200": { "description": "Lista de livros" }
        }
        },
        "post": {
        "summary": "Cadastra um livro",
        "requestBody": {
        "required": true,
        "content": {
        "application/json": {
        "schema": { "$ref": "#/components/schemas/LivroNovo" }
        }
        }
        },
        "responses": {
        "201": { "description": "Livro cadastrado" },
        "400": { "description": "Dados inválidos" }
        }
        }
        }
    },
    "components": {
        "schemas": {
        "LivroNovo": {
        "type": "object",
        "required": ["titulo", "autor"],
        "properties": {
        "titulo": { "type": "string" },
        "autor": { "type": "string" }
        }
        }
        }
```

```
  }
}
```

Fala: “O arquivo é um JSON, igualzinho ao que você aprendeu no módulo 3. No topo, a versão do padrão e as informações da API: nome, versão e uma descrição. Depois vem paths, os caminhos. Para o caminho barra livros, descrevo o método get: um resumo, o parâmetro autor, que vem na query string, e a resposta 200. E o método post: o corpo é obrigatório, em JSON, no formato de um livro novo, e as respostas possíveis são 201 e 400. O formato do livro novo está descrito mais embaixo, em components, para podermos reaproveitá-lo: um objeto com título e autor, ambos textos e obrigatórios. Na vida real, descrevemos todas as rotas. No repositório está o arquivo completo.”

[4:30] Testando no navegador

Na tela: abrir http://localhost:3000/docs; abrir o GET /livros, clicar em Try it out e em Execute, e mostrar a resposta, o código 200 e o comando curl gerado. Depois, no POST, enviar um corpo sem título e mostrar o 400.

Fala: “Vou abrir barra docs. Aqui está a nossa API documentada. Clico em GET barra livros, Try it out, Execute. A página fez o pedido de verdade e mostra a resposta, o código 200 e até o comando curl equivalente. Agora, no POST, vou enviar um corpo sem título: 400 e a mensagem. Qualquer pessoa pode explorar a API sem escrever uma linha de código.”

[5:35] Resumo

Na tela: três frases curtas.

Fala: “Resumindo: a documentação descreve o que a API oferece. O OpenAPI é o padrão do arquivo, e o Swagger UI transforma esse arquivo em uma página onde a gente também testa as rotas. No último vídeo, vamos falar do que precisa de cuidado antes de compartilhar uma API com o mundo.”

Exercício sugerido: descrever no openapi.json a rota GET /livros/{id}, com as respostas 200 e 404. Dica: no OpenAPI, o parâmetro de rota se escreve com chaves, e não com dois pontos, e o parâmetro é declarado com "in": "path" e "required": true.